

Software Development Life Cycle

- Problem Definition
- Analysis
- Design
- Development
- Testing
- Implementation
- Maintenance

1

1

Software Development Life Cycle

- Planning
- Analysis
- Design
- Implementation
- Support

2

2

Process: A Generic View

3

3

Process

A series of predictable steps
used as a road map to create a
timely, high-quality product

Product are programs,
documents, and data produced
as a consequence of the
software engineering activities
defined by the process.

4

4

Software Process

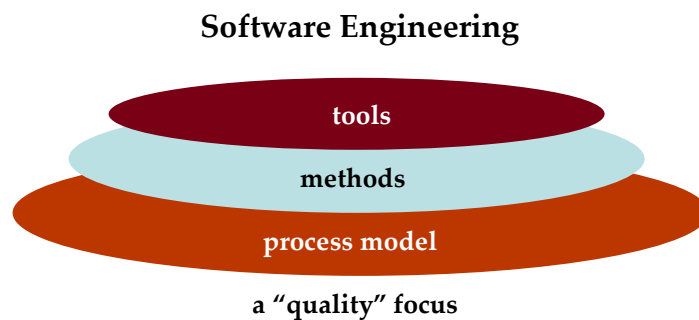
a *software process* is a framework for the tasks that are required to build high-quality software

- A structured set of activities required to develop a software system
 - Specification
 - Design and Implementation
 - Validation
 - Evolution

5

5

A Layered Technology



6

6

Software Quality

- software quality measures how well software is designed (*quality of design*), and how well the software conforms to that design (*quality of conformance*)
- Whereas *quality of conformance* is concerned with implementation, *quality of design* measures how valid the design and requirements are in creating a worthwhile product

7

7

Process model

An abstract representation of a process. It presents a description of a process from some particular perspective

8

8

Software method or methodology

The study of how to navigate through each phase of the software process model (determining data, control, partitioning functions, and allocating requirements) and how to represent phase products (structure charts, state diagram)

- Structured approach
- Object-Oriented approach

9

9

Tools

- Computer-aided software engineering (CASE) is software to support software development and evolution processes
- Activity automation
 - Graphical editors for system model development
 - Data dictionary to manage design entities
 - Graphical UI builder for user interface construction
 - Debuggers to support program fault finding
 - Automated translators to generate new versions of a program

10

10

Prescriptive Process Models

11

11

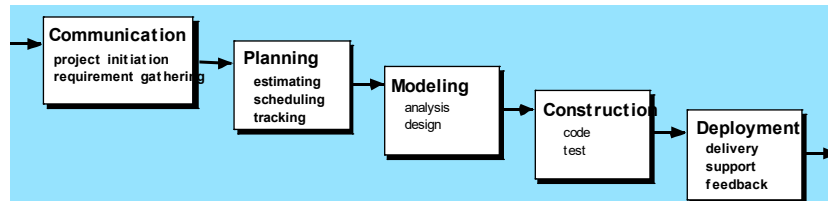
The Waterfall Model

- The waterfall model - the linear sequential model, also called the “classic life cycle”
 - > a systematic, sequential approach to software development that begins at the system level and progress through analysis, design, coding, testing, and maintenance.
- The waterfall model is the oldest and most widely used paradigm for software engineering.

12

12

The Waterfall Model



13

13

The Waterfall Model

Advantages: Simple, step-by-step, focused, and easy to follow.

Problems:

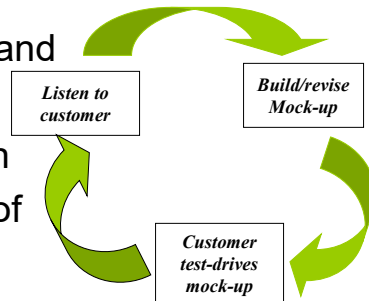
- Inflexible because real projects rarely follow the sequential flow that the model proposes.
- It is often difficult for the customers to state all requirements explicitly.
- The customer must have patience to wait to validate the software product in the late phases. (until programs are implemented)
- Customers are involved the project in the beginning.
- Developers are often delayed unnecessarily between phases.

14

14

The Prototyping Model

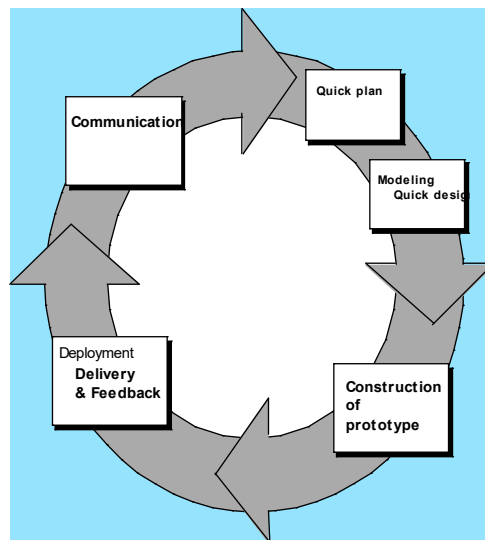
- step 1: Requirements gathering
- step 2: A “quick design” -> focuses on visible functions and behaviors of the product
- step 3: Prototype construction
- step 4: Customer evaluation of the prototype
loop back step 1.



15

15

The Prototyping Model



16

16

The Prototyping Model

Advantages:

- Easy and quick to identify customer requirements
- Customers can validate the prototype at the earlier stage and provide their inputs and feedback
- Good to deal with the following cases:
 - Customer can not provide the detailed requirements
 - Very complicated system-user interactions
 - Use new technologies, hardware and algorithms
 - Develop new domain application systems

17

17

The Prototyping Model

Problems:

- The prototype can serve as “the first system”. we should throw away.
- Developers usually attempt to develop the product based on the prototype.
- Developers often makes implementation compromises in order to get a prototyping working quickly.
- Customers may be unaware that the prototype is not a product, which is held with .

18

18

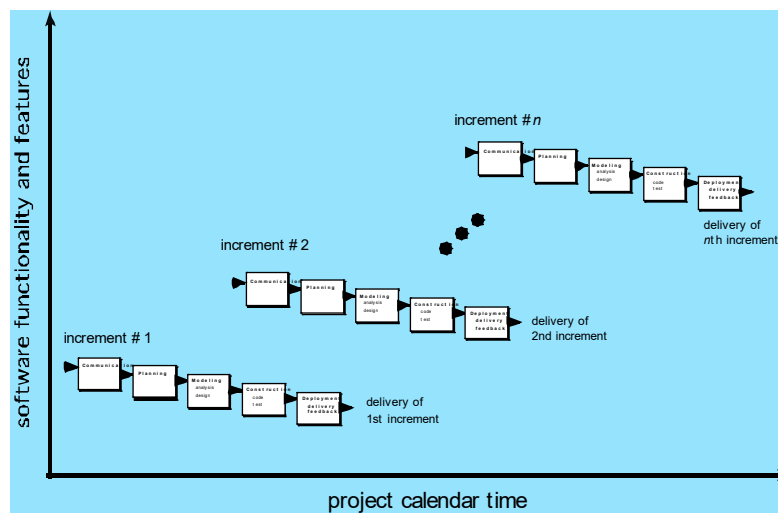
The Incremental Model

- The Incremental Model combines elements of the linear sequential model with the iterative philosophy of prototyping. The first increment is a core product.
- The incremental process model:
 - Iterative in nature, like prototyping.
 - Focus on the delivery of a operational product with each increment.
 - Particular useful when staffing is unavailable for a complete implementation by the business deadline.

19

19

The Incremental Model



20

20

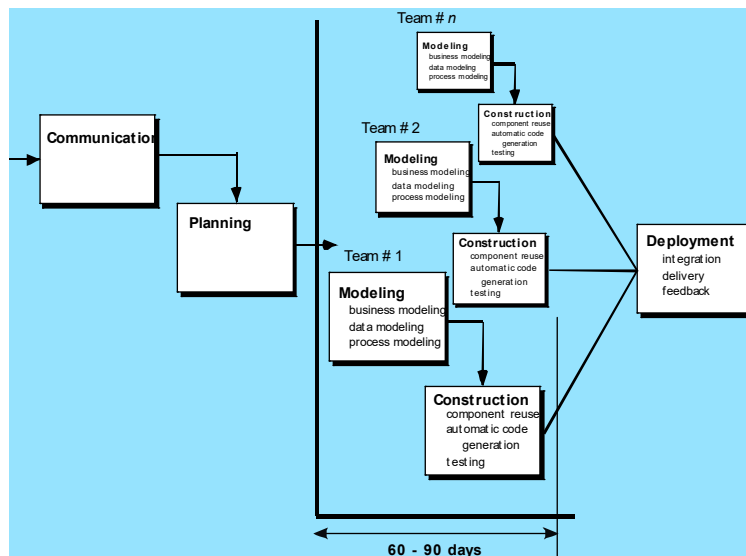
The RAD Model

- Rapid Application Development (RAD) is a linear sequential software development process model that emphasize an extremely short development cycle.
- A “high-speed” adaptation of linear sequential model
- Component-based construction
- Effective when requirements are well understood and project scope is constrained.

21

21

The RAD Model



22

22

The RAD Model

Advantages:

- Short development time
- Cost reduction due to software reuse and component-based construction

23

23

The RAD Model

Problems:

- For large, but scalable projects, RAD requires sufficient resources.
- RAD requires developers and customers who are committed to the schedule.
- Constructed software are project-specific, and may not be well modularized.
- its quality depends on the quality of existing components.
- Not appropriate projects with high technical risk and new technologies.

24

24

The Spiral Model

- The spiral model
 - is an evolutionary software process model
 - couples the iterative nature of prototyping
 - include the systematic aspects of the linear sequential model.
 - provides the potential for rapid development of incremental versions of the software.

25

25

The Spiral Model

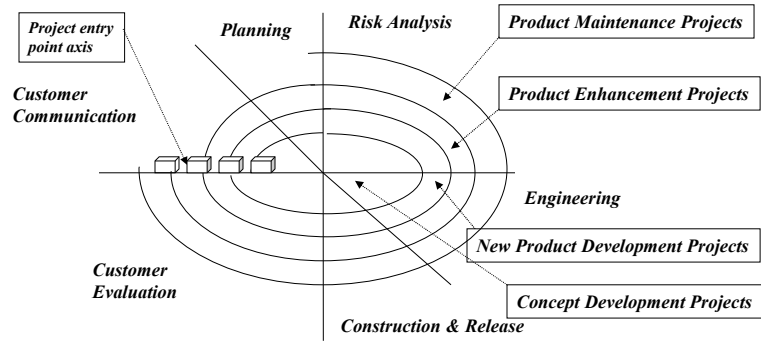
The spiral model is divided into a number of framework activities (regions):

- customer communication
 - planning (resources, timelines, etc.)
 - risk analysis
 - engineering
 - construction & release
 - customer evaluation
- Each region is populated by a series of work tasks.

26

26

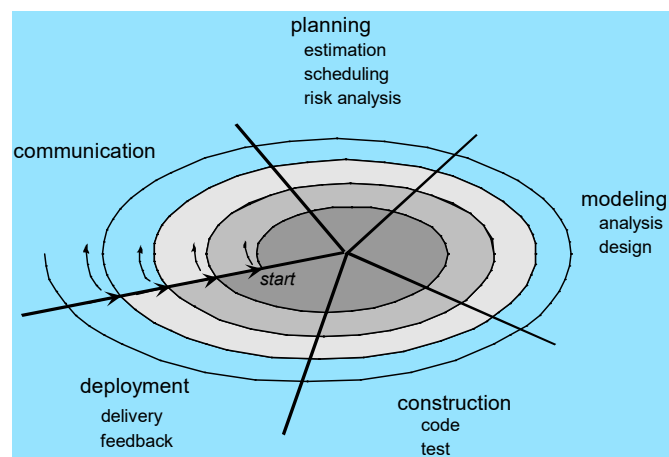
The Spiral Model



27

27

The Spiral Model



28

28

The Spiral Model

The spiral model

- a realistic approach to the development of large scale systems and software.
- may be difficult to convince customers that the evolutionary approach is controllable.
- a relatively new model, and has not used as widely as the linear sequential or prototyping paradigms.

29

29

The Concurrent Model

- The concurrent development model - called concurrent engineering.
- > provides an accurate state of the current state of a project.
- Focus on concurrent engineering activities in a software engineering process such as prototyping, analysis modeling, requirements specification and design.
- Represented schematically as a series of major technical activities, tasks, and their associated states.

30

30

The Concurrent Model

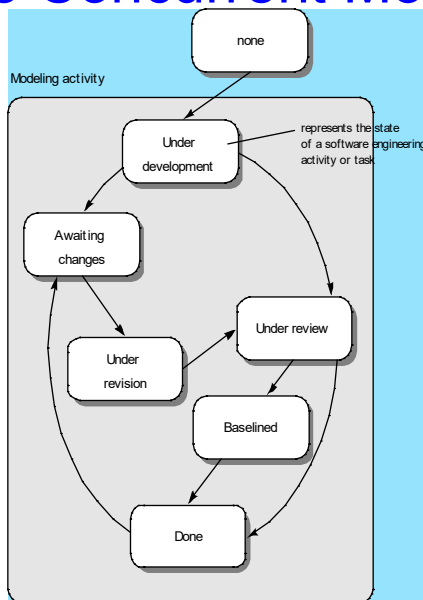
- Defined as a series of events that trigger transitions from state to state for each of the software engineering activities.
- Two ways to achieve the concurrency:
 - system and component activities occur simultaneously and can be modeling using the state-oriented approach
 - a typical client/server application is implemented with many components, each can be designed and realized concurrently.

Application: all types of software development

31

31

The Concurrent Model



32

32

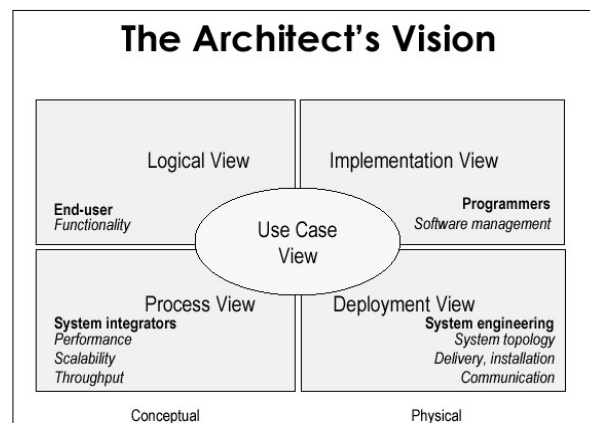
The Unified Process (UP)

- Use case-driven
- Incremental and Iterative development
- Architecture-centric
- Visual Modeling using UML

33

33

The Unified Process (UP)



34

34

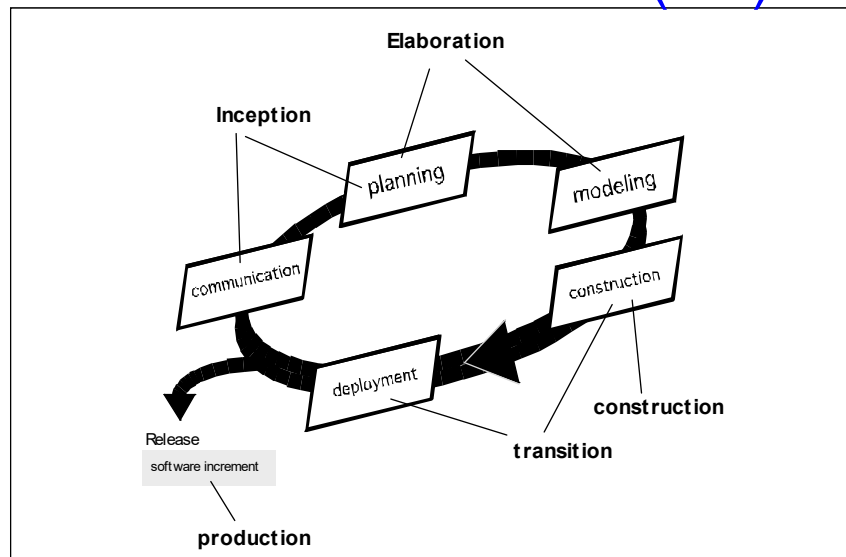
The Unified Process (UP)

- Inception
 - Return of investment
 - Feasibility study
 - Features and software requirement
 - Plan
- Elaboration
 - Architectural representation: use case model, analysis model, design model,
- Construction
 - Initial operational Capability: coding, testing
- Transition
 - Product Release: beta testing, support information (e.g., user manuals)
- Production
 - The on-going use of the software is monitored. Support for the operating environment is provided, and defect reports and requests for change are submitted and evaluated.

35

35

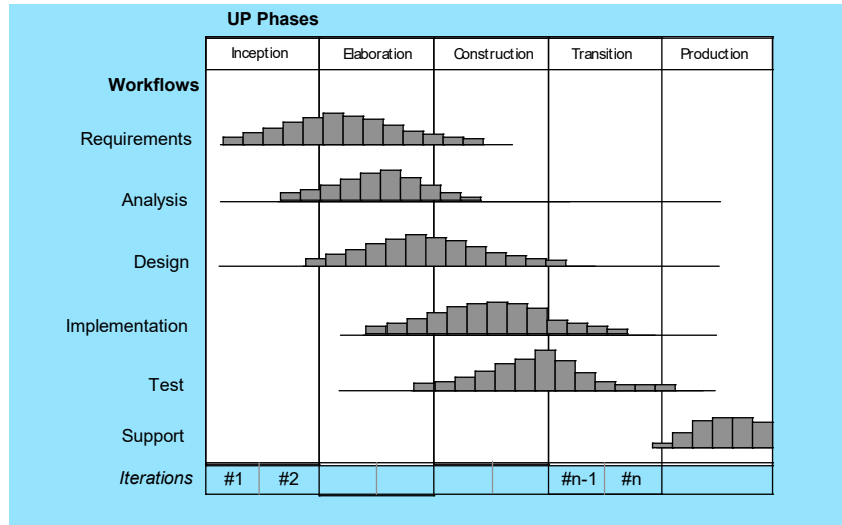
The Unified Process (UP)



36

36

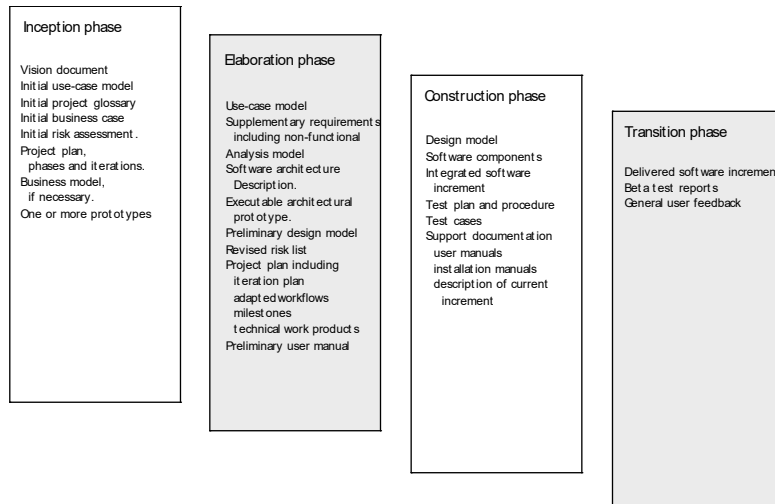
UP Phases



37

37

UP Work Products



38

38

Umbrella Activities

An umbrella activity is a support activity that spans the entire life cycle of a software project and is essential for successful project execution.

- Software project management
- Formal technical reviews
- Software quality assurance
- Software configuration management
- Work product preparation and production
- Reusability management
- Measurement
- Risk management

39

39

The CMMI

- The Capability Maturity Model Integration
- Software Engineering Institute (SEI), Carnegie Mellon, USA
- CMMI is a process improvement approach that provides organizations with the essential elements of effective processes.

40

40

Category	Process Area
Project Management	•Project Planning •Project Monitoring and Control •Supplier Agreement Management •Integrated Project Management (IPPD) •Integrated Supplier Management (SS) •Integrated Teaming (IPPD) •Risk Management Quantitative Project Management
Support	•Configuration Management •Process and Product Quality Assurance •Measurement and Analysis Causal Analysis and Resolution •Decision Analysis and Resolution •Organizational Environment for Integration (IPPD)
Engineering	•Requirements Management •Requirements Development •Technical Solution •Product Integration •Verification •Validation
Process Management	•Organizational Process Focus •Organizational Process Definition •Organizational Training •Organizational Process Performance •Organizational Innovation and Deployment

41

41

The CMMI

Capability levels

0 Incomplete

- An "incomplete process" is a process that is either not performed or partially performed.

1 Performed

- A process that is expected to perform all of the Capability Level 1 specific and generic practices. Performance may not be stable and may not meet specific objectives such as quality, cost, and schedule, but useful work can be done.

2 Managed

- A managed process is planned, performed, monitored, and controlled for individual projects, groups, or stand-alone processes to achieve a given purpose.

42

42

The CMMI

Capability levels

3 Defined

- A defined process is a managed (capability level 2) process that is tailored from the organization's set of standard processes according to the organization's tailoring guidelines, and contributes work products, measures, and other process-improvement information to the organizational process assets.

4 Quantitatively Managed

- A quantitatively managed process is a defined (capability level 3) process that is controlled using statistical and other quantitative techniques.

5 Optimizing

- An optimizing process is a quantitatively managed process that is improved, based on an understanding of the common causes of process variation inherent in the process.

43

43

The CMMI

Maturity levels

1 Initial

- organizations often produce products and services that work; however, they frequently exceed the budget and schedule of their projects.
- Maturity level 1 organizations are characterized by a tendency to over commit, abandon processes in the time of crisis, and not be able to repeat their past successes.

2 Managed

- requirements, processes, work products, and services are managed. The status of the work products and the delivery of services are visible to management at defined points.
- Commitments are established among relevant stakeholders and are revised as needed. Work products are reviewed with stakeholders and are controlled.
- The work products and services satisfy their specified requirements, standards, and objectives.

44

44

The CMMI

Maturity levels

3 Defined

- processes are well characterized and understood, and are described in standards, procedures, tools, and methods

4 Quantitatively Managed

- Quantitative objectives for quality and process performance are established and used as criteria in managing processes. Quantitative objectives are based on the needs of the customer, end users, organization, and process implementers. Quality and process performance are understood in statistical terms and are managed throughout the life of the processes.

45

45

The CMMI

Maturity levels

5 Optimizing

Processes are continually improved based on a quantitative understanding of the common causes of variation inherent in processes.

46

46

The CMMI

Process Area (PA):

Maturity Level 2

1. Configuration Management
2. Measurement Analysis
3. Project Monitoring and Control
4. Project Planning
5. Process and Product Quality
6. Requirement Management
7. Supplier Agreement Management

47

47

Q&A

48

48